Numerisches Lösen

PROJEKTARBEIT ZUM BISEKTIONSVERFAHREN UND NEWTON-VERFAHREN

Hadžiaganović Laila
3BHWI | SWP | 2025/26

Numerisches Lösen – Bisektionsverfahren

Aufgabe 1)

Mathematische Umformung

$$x^2 = 4 \rightarrow f(x) = x^2 - 4$$

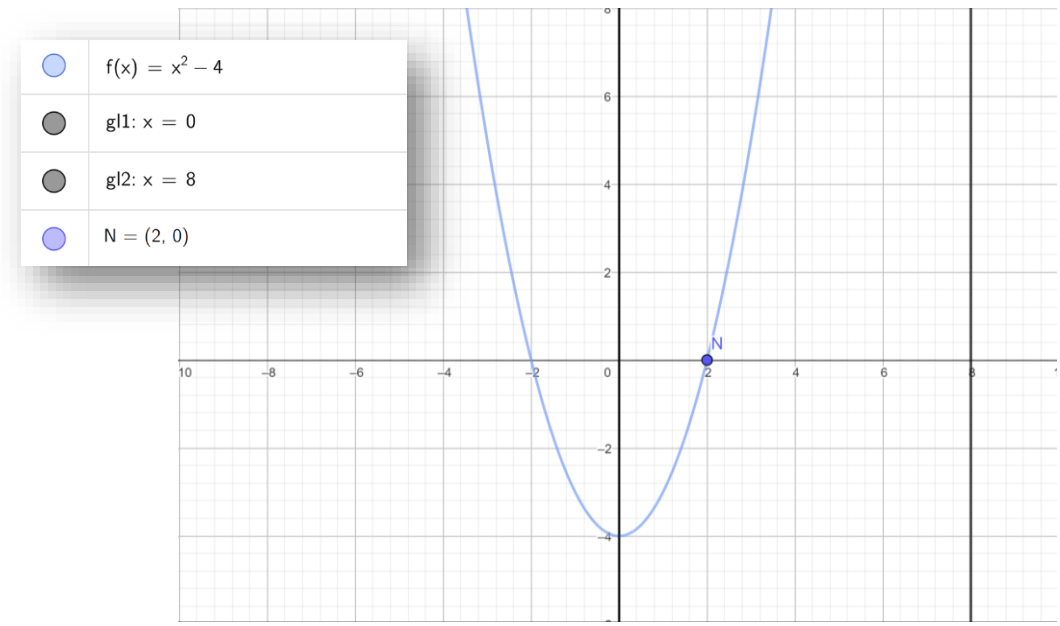
Nachweis des Vorzeichenwechsels

Intervall = [0, 8] $(2 * n = 8)$

Untere Grenze (a = 0) $f(0) = 0^2 - 4 = \mathbf{-4}$ $(negativ)$

Obere Grenze (b = 8) $f(8) = 8^2 - 4 = \mathbf{60}$ $(positiv)$

Da $f(0) < 0$ und $f(8) > 0$ gilt, muss für stetige Funktionen mindestens eine Nullstelle im Intervall [0, 8] existieren. *(Zwischenwertsatz)*



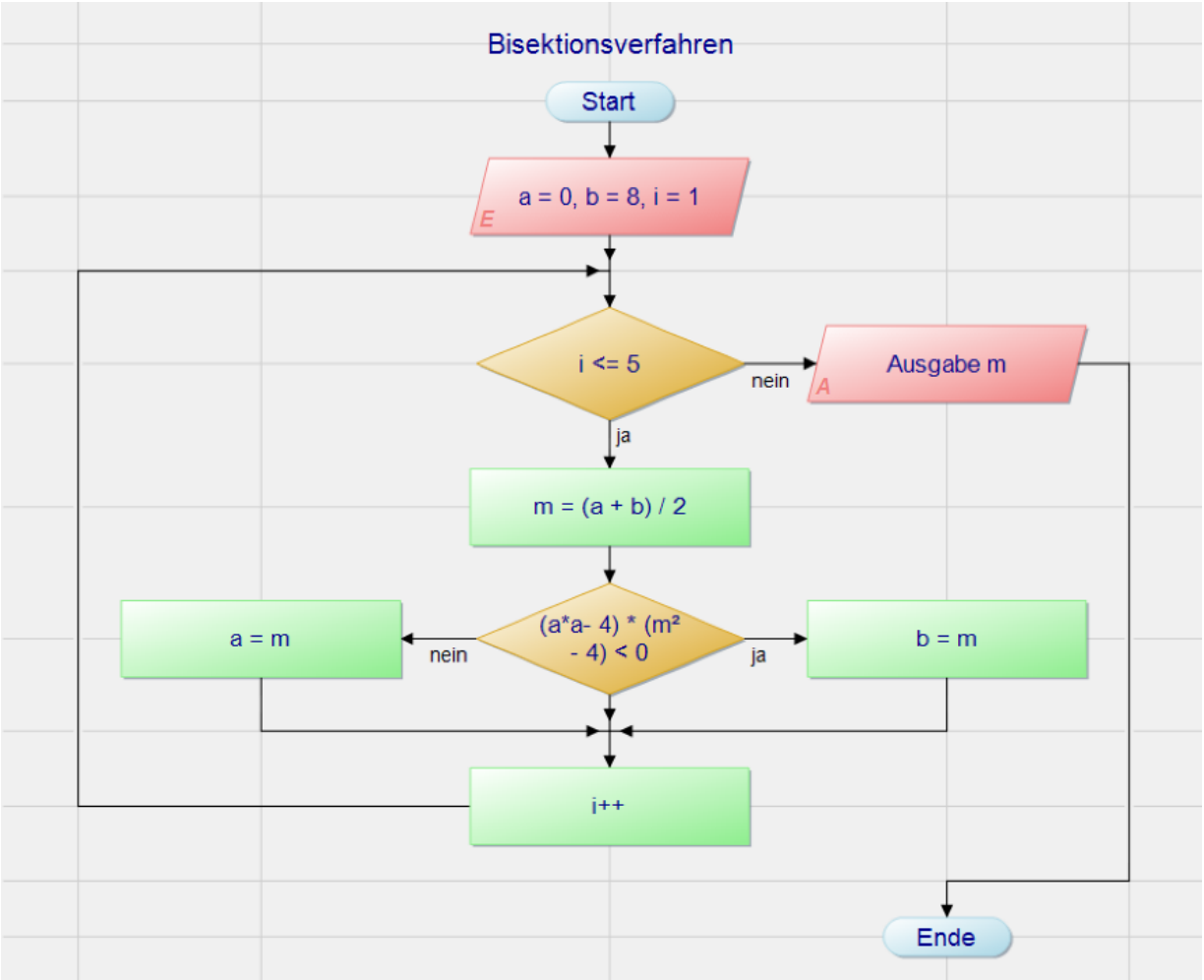
Da das Polynom auf seinem gesamten Definitionsbereich **stetig** ist (keine Sprünge oder Lücken) und ein **Vorzeichenwechsel** im Intervall vorliegt, ist das **Bisektionsverfahren** anwendbar.

Aufgabe 2)

Bisektionsverfahren tabellarisch

Schritte	a	b	$Mitte\ m = \frac{a+b}{2}$	$f(m) = m^2 - 4$	Vorzeichen $f(m)$	Intervall $_{neu}$
1	0	8	4	$4^2 - 4 = 12$	positiv (+)	[0, 4]
2	0	4	2	$2^2 - 4 = 0$	Nullstelle	[2, 2]
3	2	2	2	$2^2 - 4 = 0$	0	[2, 2]
4	2	2	2	$2^2 - 4 = 0$	0	[2, 2]
5	2	2	2	$2^2 - 4 = 0$	0	[2, 2]

Aufgabe 3)



Aufgabe 4)

Vorgehensmodell: Das Wasserfallmodel

Warum?

Das Verfahren ist eine klar definierte mathematische Aufgabe, daher gibt es keine Anforderungen, die sich währenddessen ändern. Ich plane erst, erstelle ein PAP und „teste“ am Ende mithilfe des Programmierens. Dies ist ein linearer Prozess, wodurch sich das Wasserfallmodel perfekt dazu eignet.

Programmier-Paradigma: Strukturiert

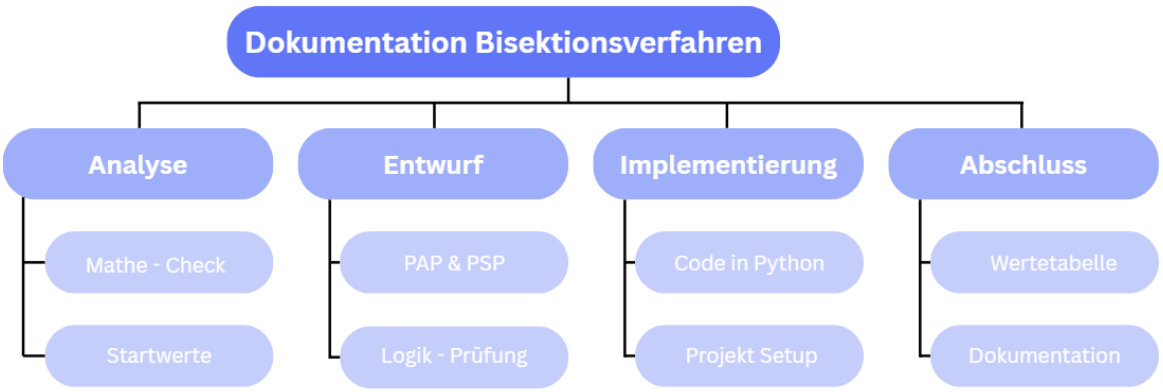
Warum?

Ich wähle das strukturierte Programmier-Paradigma. Der Fokus liegt auf dem logischen Ablauf (Sequenz, Verzweigung und Wiederholung), der sich durch Kontrollstrukturen wie Schleifen und Bedingungen abbilden lässt.

Zeitmanagement

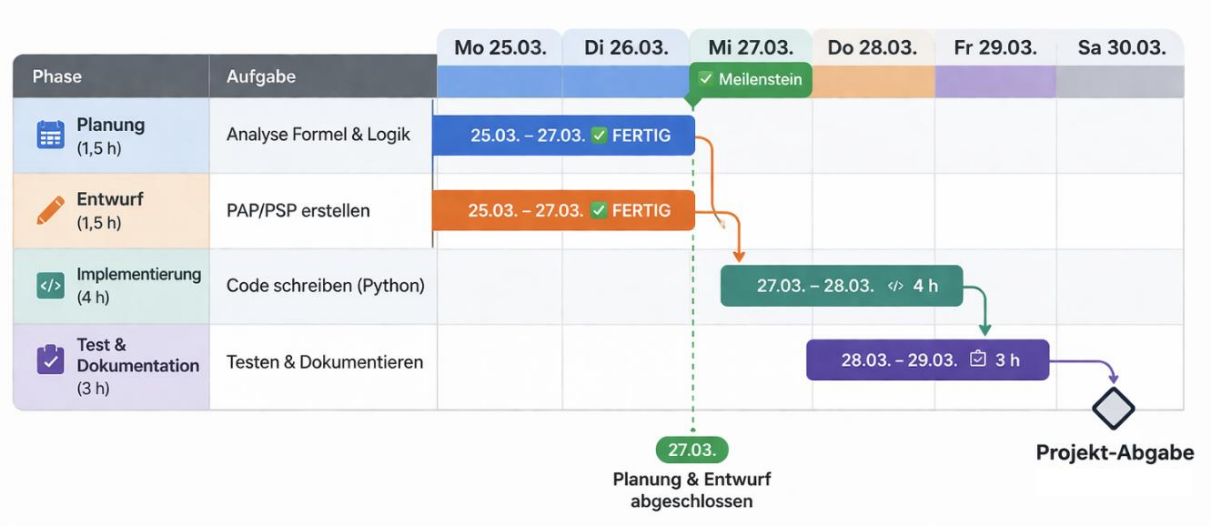
Phase	Tätigkeit	Geschätzte Dauer
Planung	Analyse Formel & Logik	1,5 h
Entwurf	Erstellung des PAP/PSP	1,5 h
Implementierung	Schreiben des Codes	4 h
Test & Dokumentation	Prüfen & Dokumentation	3 h
Gesamt		10 h

WBS / PSP



Gantt Diagramm

Soll und Ist – Zustand:



Ressourcenmanagement

Personal: 1 Software -Entwickler (Laila H.) für Planung, Codierung und Test

Hardware:

- Standard – **Laptop** (Tastatur, Display) & Prozessor und RAM-Speicher!!
- **Internetverbindung** für Recherche und Software – Downloads

Software:

- **Betriebssystem:** Windows 11
- **Entwicklungsumgebung:** Visual Studio Code für Python (3.x)
- **Standard-Python:** keine externen Bibliotheken nötig
- **Planungtools:** PapDesigner, Microsoft Word, Canva (PSP & Gantt)

Risikomanagement:

Innere Risiken:

- **Logik- und Denkfehler**
  - **Risiko:** mathematische Algorithmus falsch implementiert
  - **Gegenmaßnahme:** Händische Durchführung & Abgleich

- **Fehlkalkulation der Zeit**
  - **Risiko:** Arbeitspakete nehmen mehr Zeit ein als geplant
  - **Gegenmaßnahme:** Einplanung von zeitlichen Puffern

Äußere Risiken:

- **Technischer Defekt**
  - **Risiko:** Hardware-Schaden am Laptop/Software-Absturz
  - **Gegenmaßnahme:** Backup-Strategie (extern)

- **Persönlicher Ausfall**
  - **Risiko:** Krankheit oder unvorhersehbare Ereignisse
  - **Gegenmaßnahme:** Früherer Projektstart weit vor Abgabetermin

Veröffentlichungsprüfung:

PSF-Lizenz: Da ich Python nutze, unterliegt mein Projekt indirekt der ***Python Software Foundation License,*** jedoch bleibe ICH der Urheber.

Liberal & frei: Ich darf den Code verbreiten, verändern und sogar verkaufen, solange der Urheberrechtshinweis von Python erhalten bleibt.

Die Software ist rechtlich unbedenklich und darf unter der PSF-Lizenz oder als Open Source veröffentlicht werden.

Markenrecht: Python und Visual Studio Code sind geschützte Marken der jeweiligen Organisation; Programm selbst neutraler Name; keine Verwechslungsgefahr besteht mit offiziellen Produkten der Markeninhaber.

Aufgabe 5)

<u>Programmierung des Bisektions-Verfahrens</u>

Die Implementierung des **Bisektionsverfahrens** erfolgte in der Datei *solver_bisektion.py*. Um den Anforderungen gerecht zu werden, wurde der Code in Funktionsblöcke unterteilt. Die zentrale bisektion-Funktion arbeitet als generischer Solver, der eine Callable-Funktion als Parameter entgegennimmt. Dies ermöglicht eine **hohe Wiederverwendbarkeit** für beliebige mathematische Funktionen. Zur Sicherstellung wurden Fehlerbehandlungen mittels try-except-Blöcken implementiert, die beispielsweise ungültige Intervallgrenzen abfangen.

Die mathematische Logik hinter dem Verfahren basiert auf der wiederholten Halbierung des Intervalls $[a, b]$, wobei in jedem Schritt durch Vorzeichenprüfung von $f(a) \cdot f(m)$ entschieden wird, in welcher Hälfte die Nullstelle liegt, bis die gewünschte Genauigkeit (10^{-8}) erreicht ist.

```
test vom bisektionsverfahren
----------------------------------------

wert n = 25
numerische lösung : 5.00000000174623
analytische lösung: 5.0
abweichung        : 1.7462298274040222e-09

wert n = 81
numerische lösung : 8.999999996332917
analytische lösung: 9.0
abweichung        : 3.6670826375484467e-09

wert n = 144
numerische lösung : 12.000000001396984
analytische lösung: 12.0
abweichung        : 1.3969838619232178e-09
PS C:\Users\laila\OneDrive - tgm - Die Schule
I\bisektion-projekt>
```

Aufgabe 6)

<u>Erweiterung der Lösungsverfahren</u>

Die alternative Lösungsmöglichkeit wurde mit dem **Newton-Raphson-Verfahren** umgesetzt. Dieses Verfahren bestimmt eine Nullstelle iterativ, indem ausgehend von einem Startwert mit Hilfe der Ableitung der Funktion schrittweise eine bessere Näherung berechnet wird.

Die Implementierung erfolgte in der Datei *solver_newton.py*. Der Code wurde wieder in einzelne Funktionsblöcke unterteilt, damit die Lösung übersichtlich und wiederverwendbar bleibt. Die Newton-Funktion erhält die zu lösende Funktion, ihre Ableitung und einen Startwert als Parameter. Dadurch kann das Verfahren auch für andere Funktionen verwendet werden.

Die Überprüfung des Programms erfolgte wieder mit der Wurzel-Funktion aus Aufgabe 1 für die Werte 25, 81 und 144. Die numerischen Ergebnisse stimmen mit den analytischen Lösungen überein, wodurch die Korrektheit des Newton-Raphson-Verfahrens bestätigt wird.

```
test vom newton-verfahren
----------------------------------------

wert n = 25
numerische lösung : 5.0
analytische lösung: 5.0
abweichung        : 0.0

wert n = 81
numerische lösung : 9.0
analytische lösung: 9.0
abweichung        : 0.0

wert n = 144
numerische lösung : 12.0
analytische lösung: 12.0
abweichung        : 0.0
PS C:\Users\laila\OneDrive - tgm - Die Sch
I\bisektion-projekt>
```

Im Vergleich zum Bisektionsverfahren konvergiert das Newton-Verfahren schneller, **benötigt jedoch zusätzlich die Ableitung der Funktion**.

Aufgabe 7)

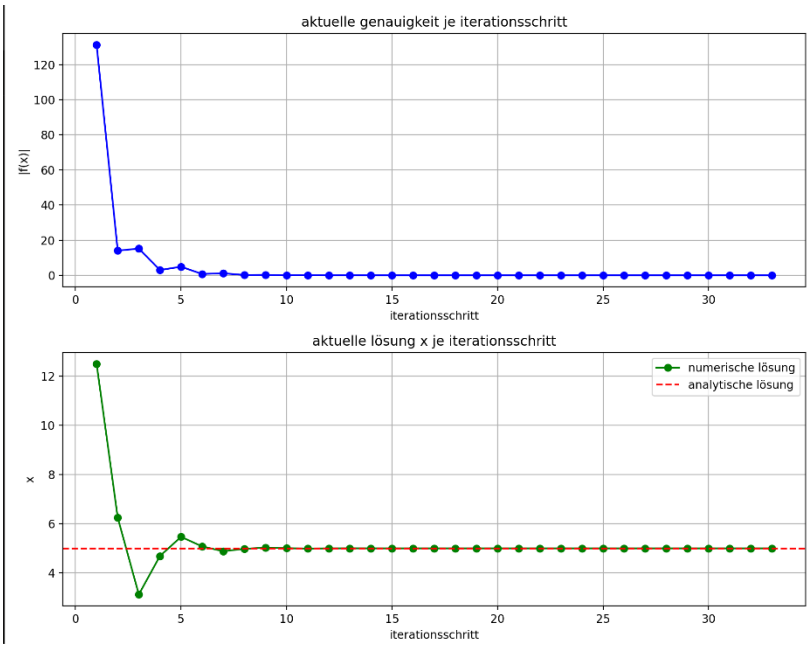
<u>Grafische Aufbereitung (Matplotlib)</u>

Die grafische Aufbereitung der Nullstellenfindung erfolgt mit der Bibliothek Matplotlib. Dazu wurde ein eigenes PythonProgramm erstellt, das während des Bisektionsverfahrens die aktuellen Werte jeder Iteration speichert und anschließend grafisch darstellt.

Mit Hilfe von Subplots wurden zwei Diagramme in einer Figure ausgegeben. Im ersten Diagramm wird die aktuelle Genauigkeit je Iterationsschritt dargestellt. Dafür wurde der Wert | $f(x)$ | verwendet, welcher sich im Laufe der Iteration immer weiter dem Wert 0 annähert.

Im zweiten Diagramm wird die aktuelle Lösung x je Iterationsschritt gezeigt. Dadurch ist erkennbar, wie sich die numerisch berechnete Lösung schrittweise der analytischen Lösung annähert. Zusätzlich wurde die analytische Lösung als horizontale Vergleichslinie eingezeichnet.

Die grafische Darstellung macht den iterativen Verlauf des Verfahrens gut sichtbar und bestätigt die korrekte Arbeitsweise des Programms:



Aufgabe 8)

<u>Validierung & Genauigkeitstests</u>

Zur Überprüfung der Genauigkeit wurde der Solver mit dem Polynom $P_4(x) = 2x + x^2 + 3x^3 - x^4$ getestet. Für die gesuchte Nullstelle bei $x = 3{,}4567$ wurde das Intervall $[3, 4]$ gewählt, da in diesem Bereich ein Vorzeichenwechsel und damit eine Nullstelle vorliegt.

Anschließend wurde untersucht, wie viele Iterationsschritte für die Genauigkeiten 10^{-2} und 10^{-8} notwendig sind. Die Tests im Terminal zeigen, dass für 10^{-2} insgesamt 7 Iterationen und für 10^{-8} insgesamt 27 Iterationen benötigt wurden.

Die Ergebnisse bestätigen die korrekte Arbeitsweise des Solvers. Außerdem ist erkennbar, dass mit steigender Genauigkeit auch mehr Iterationsschritte notwendig sind.

Genauigkeit	Iterationsschritte
10^{-2}	7
10^{-8}	27

```
I\bisektion-projekt> python solver_test_polynom.py
genauigkeit 0.01: 7 iterationen
genauigkeit 1e-08: 27 iterationen
```

Aufgabe 9)

<u>Anwendung am realen Problem (Kettenlinie)</u>

Problemstellung:

Die Aufgabe besteht darin, die Länge l einer elektrischen Leitung zwischen zwei Masten zu berechnen. Die Formel für die Kettenlinie $y(x) = a \cdot \cosh\left(\frac{x}{a}\right) - a + y_0$ ist transzendent, da der Krümmungsradius a nicht isoliert werden kann. Zur Lösung muss daher ein numerisches Verfahren angewendet werden!

Mathematischer Ansatz:
Unter Berücksichtigung der Randbedingungen (Abstand 100m, Durchhang 10m) ergibt sich bei $x = 50$ die Bedingung:

$$a \cdot \cosh\left(\frac{50}{a}\right) - a = 10$$

Dies entspricht der Nullstellen-Suche für $f(a) = a \cdot \cosh\left(\frac{50}{a}\right) - a - 10 = 0$.

Implementierung
Zur Lösung wurde das bereits in Aufgabe 5 / 6 entwickelte **Bisektionsverfahren** eingesetzt. Im Code wird im Intervall $[50, 200]$ iterativ nach dem Krümmungsradius a gesucht. Nach Bestimmung von a wird die Leitungslänge mittels der Formel $l = 2a \cdot \sin\mathrm{h}\left(\frac{100}{2a}\right)$berechnet.

Ergebnisse
Die numerische Berechnung liefert folgende Werte:

- **Krümmungsradius** a**:** $126{,}6324$ m
- **Leitungslänge** l**:** $102{,}6187$ m

Die Ergebnisse bestätigen die Genauigkeit des numerischen Lösungsansatzes:

```
I\bisektion-projekt> python solver_kettenlinie.py
krümmungsradius a: 126.6324 m
leitungslänge l: 102.6187 m
```